

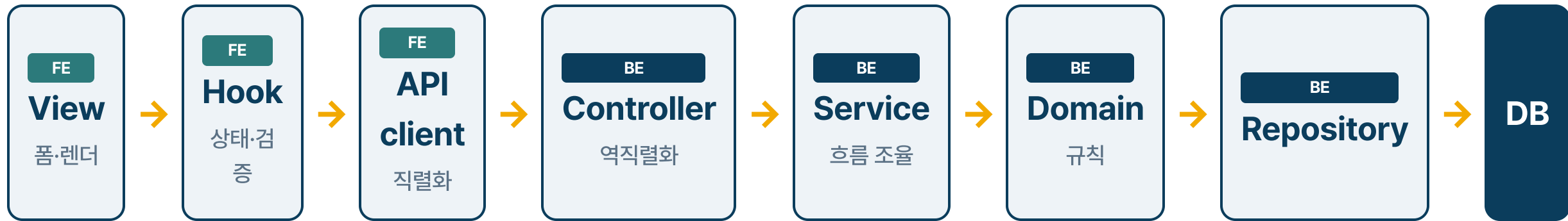
데이터는 어떻게 흐르고 어떻게 변형되는가

보험 시스템 — 웹 프론트엔드에서 데이터베이스까지

분산프로그래밍 기말 발표 · 2026-06-09

1. 시스템 개요와 컴포넌트 지도

한 번의 요청이 거치는 전체 경로입니다. 모든 도메인이 동일한 계층 구조를 따릅니다.



- **프론트엔드:** React, feature 단위(View → Hook → API client)로 구성
- **백엔드:** Spring, 도메인별 Controller → Service → Domain → Repository
- **경계마다 데이터의 표현 형태가 바뀝니다.** 오늘의 핵심은 그 변형 지점입니다.

발표의 초점은 기능이 아니라 데이터의 흐름과 변형입니다.

같은 정보가 계층을 지날 때마다 어떤 형태로 바뀌는지, 그리고 그 변형을 어떻게 통제했는지를 보입니다.

2. 데이터 흐름 ① — 가입 신청 (요청 경로)

ProductsView

FE · 컴포넌트

폼 입력 (FormData, name=*)

useCustomerContracts

FE · 훅

JS 객체 CreateApplicationRequest { productId, vehicleInfo?, medicalHistory? }

contractsApi.createApplication

FE · API

JSON.stringify → HTTP 본문 + Bearer 토큰

POST /applications

네트워크

application/json 요청

ApplicationController.apply

BE · 컨트롤러

@Valid @RequestBody record + @AuthenticationPrincipal userId

request.toVehicleInfo() / toMedicalHistory()

BE · 변환

도메인 값 객체 VehicleInfo / MedicalHistory

ApplicationService.apply

BE · 서비스

new InsuranceApplication(applicant, product, ...)

Repository.save

INSERT 자동 생성 → DB row

2. 데이터 흐름 ① — 응답 경로와 변형 요약

InsuranceApplication

BE · 엔티티

```
status=PENDING, appliedAt=now()
```

ApplicationResponse.from(app)

BE · DTO 변환

```
{ applicationId, status, appliedAt }
```

HTTP 201

네트워크

응답 JSON

contractsApi → 혹

FE

TS 타입 ApplicationCreated

ProductsView

FE · 렌더

성공 메시지 + 목록 재조회(MyApplication[])

하나의 신청 정보가 형태를 7번 바꿉니다. 폼 → JS 객체 → JSON → 요청 record → 도메인 객체 → DB row → 응답 record → JSON → 화면. 경계마다 변환 책임이 명확히 한 곳에 있습니다.

3. 데이터 흐름 ② — 입력 한 개가 계약을 바꾼다 (도메인 규칙)

직원이 입력한 **할증률** 하나가 계약의 **월 보험료**로 변형되는 과정입니다.

EmployeeReviewsPage

FE

controlled state: result, comment, surchargeRate

underwritingApi.confirmReview

FE

POST /reviews/applications/{id}/confirm – ConfirmReviewRequest

ReviewService.confirm

BE · 서비스

EnrollmentReview.confirm(result, surchargeRate, basePremium)

EnrollmentReview

BE · 도메인

surchargeRate 적용 → adjustedPremium 산출

승인 시 같은 트랜잭션

BE

new InsuranceContract(보험료 = adjustedPremium) (ADR 0005·0003)

ConfirmReviewResponse

→ 화면

FE

{ adjustedPremium } → "최종 월 보험료 ...원"

화면에 보이는 "최종 월 보험료"는 직원의 입력값이 도메인 규칙을 거쳐 계약에 기록된 결과입니다. 표현이 아니라 상태 변화입니다.

4. 계층 경계와 DTO의 역할

왜 매 계층에서 형태를 바꿀까요? 각 계층의 관심사를 분리하기 위해서입니다.

경계	변환	책임
화면 ↔ 네트워크	폼/객체 ↔ JSON	직렬화 (API client)
네트워크 ↔ 백엔드	JSON ↔ Request record	검증·역직렬화 (Controller)
요청 ↔ 도메인	record ↔ 도메인 객체	의미 부여 (Service·VO)
도메인 ↔ DB	엔티티 ↔ row	영속화 (Repository/JPA)
도메인 ↔ 화면	엔티티 → Response	노출 범위 통제 (<code>from()</code>)

도메인 모델은 웹·DB 어느 쪽에도 오염되지 않습니다. Response DTO가 엔티티를 외부로부터 격리하며, 모든 도메인이 `XxxResponse.from(entity)` 정적 팩토리로 통일되어 있습니다.

5. 같은 요소가 단계별로 어떻게 변형되었나

InsuranceApplication 한 요소를 TUI/JDBC 단계 → Spring 단계로 추적했습니다.

요소	이전 (TUI · JDBC/DAO)	현재 (Spring · JPA)
식별자	applicationId : String (수기 생성)	id : Long @GeneratedValue
개인정보	holderName 등을 신청서에 복제	Policyholder 참조 (ADR 0002)
입력 검증	Scanner while-loop 수동 검증	@Valid·@NotNull + 엔티티 불변식
영속화	수기 INSERT + map(ResultSet)	repository.save() (ORM)
일시	epoch long (getTime())	LocalDateTime
표현(출력)	System.out.printf — 핸들러에 혼재	Controller가 DTO(JSON)로 분리

단순한 기술 교체가 아니라, 책임의 재배치입니다. 출력·검증·식별·개인정보 소유의 위치가 모두 바뀌었습니다.

6. 그 변형을 통제한 것 — 다이어그램 ↔ 코드 1:1

요소가 변형되어도 도메인 모델은 제가 설계한 다이어그램을 벗어나지 않았습니다. 의도된 변경은 모두 ADR로 기록한 뒤 다이어그램에 역반영했습니다.

변경	코드 현실	결정 근거
ClaimStatus 확장	+ COMPLETED, FAILED	ADR 0007
지급심사 대상 일반화	BenefitPaymentReview → Claim(추상)	ADR 0009
승인 시 계약 자동 생성	같은 트랜잭션 내 생성	ADR 0005
조건부 승인·할증	adjustedPremium 단일 필드	ADR 0003

변경의 주체는 AI가 아니라 설계자입니다. 문서가 코드를 뒤따릅니다.

7. AI 도구의 한계와 통제 — 발견과 해결

① 성급한 추상화

DB 도입 전 컨트롤러를 분리 3fc0ce3

→ 실익이 없어 되돌림 3e57d68

② 유스케이스 불일치

자동차 보상심사를 자동 배정으로 구현 bd2d7ce

→ 명세대로 수동 배정으로 정정 206bf25

③ 계층 패턴 이탈

일부 컨트롤러가 Repository를 직접 호출

→ 패턴 점검으로 식별·관리

④ 용어·도메인 오염

동의어 혼용, 도메인에 출력 코드 삽입 경향

→ 용어집·ADR로 사전 차단 (도메인 출력 0)

통제 장치: 다이어그램(설계) · CONTEXT.md(용어) · ADR(결정) · TDD(검증). AI는 구현을 가속했고, 방향은 이 장치들이 잡았습니다.

결론

데이터의 변형은 통제된 설계의 결과입니다

- 정보는 계층을 지나며 **명확히 정의된 경계에서만** 형태를 바꿉니다.
- 단계가 진화해도 **도메인 모델과 다이어그램의 일치**는 유지됩니다.
- AI는 구현을 가속했고, **설계·용어·결정·검증은 사람이** 통제했습니다.

감사합니다. 질문 받겠습니다.